

[IS216] Extra Exercises - Vue Basics II (Events)

Objectives

- Gain a solid understanding of reactive programming concepts.
- Learn to leverage Vue framework features for building efficient web applications.
- Practice applying Vue directives to create dynamic and responsive web pages.

Instructions

- Questions without an asterisk are easy.
- Questions marked with * are moderately challenging.
- Questions marked with ** are challenging.
- Questions marked with *** are very challenging.

Download

- **Resources:** Click [here](#)
- **Sample Solutions:** Click [here](#)

Contents

Question 1 - Farm (**)	2
Question 2 - Cat Food (*)	4
Question 3 - Colourful words (*)	6
Question 4 - Flip coin (**)	7
Question 5 - Temperature (*)	9
Question 6 - Calculator (*/**/***)	10
Question 7 - Monster Slayer 2 (***)	14

Note:

- If you notice any mistakes or errors in the questions, please email your instructors with a description of the issue. We will address it as soon as possible.
- For the best viewing experience, please use Microsoft Office 365 Online.

Question 1 - Farm (**)

Background:

You are tasked with developing a web-based animal farming game. The game should allow players to add animals to their farm, depending on which animals are available at the animal market each day.

Requirements:

- When the application first loads (see Figure q1a):
 - It should display two dynamically generated radio button inputs, with no default selection, using two randomly selected animal options. This can be done by creating a computed property that returns an array containing two distinct animals randomly selected from the `animals` data property (no duplicates allowed). The computed property can then be used to iterate over the array and render the radio button inputs.
 - It should also display an “Add to Farm” button.
- When an animal option is selected and the “Add to Farm” button is clicked (see Figures q1b and q1c):
 - The selected animal should be added to the farm and a text message such as “Your farm is composed by ...” should be displayed. This can be done by creating a method that pushes the selected animal into the `farm` data property. The farm property can then be used to generate a descriptive string for display.
 - Only one animal can be added to the farm per click.

Instructions:

1. Navigate to the `q1` directory in your terminal.
2. Run the command `pnpm install` to install the required dependencies.
3. Run the command `pnpm dev` to run the app in your development server.
4. After installation, navigate to the `q1/src` directory.
5. Open the `App.vue` file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- The icon representing each animal is defined in the `animals` data property.

When the application first loads.

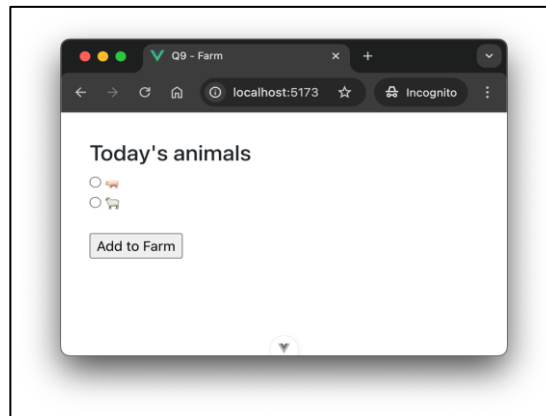


Figure q1a

When the player selects "cow" and clicks the "Add to Farm" button.

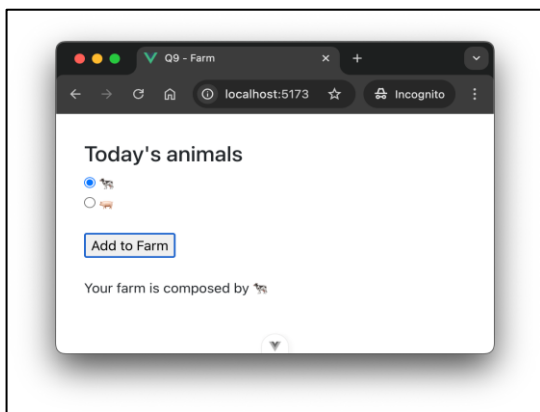


Figure q1b

When the player selects "pig" and clicks the "Add to Farm" button.

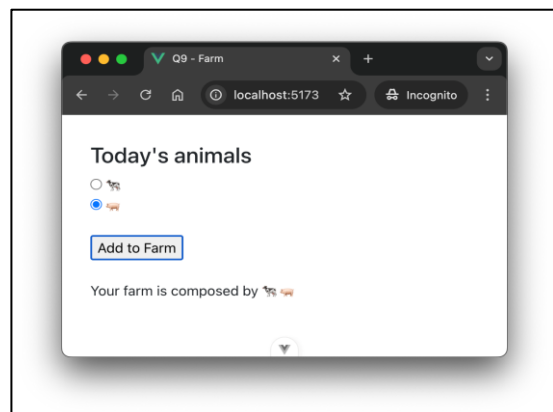


Figure q1c

Question 2 - Cat Food (*)

Background:

You are tasked with developing an e-commerce website featuring a shopping cart for selling cat food. The application should allow users to add items to the cart, but once the quantity added reaches the available inventory, the application should prevent users from adding more.

Requirements:

- When the application first loads (see Figure q2a):
 - It should display the following using the information from the Vue instance's data property:
 - The site name inside an `<h2>` tag.
 - The product image (`cat . jpg`).
 - A shopping cart icon alongside the current quantity value set to 0.
 - The product's title and description.
 - The formatted product price, which can be created using a computed property that processes the given price value by following these steps:
 - Check that the price is a positive integer; if not, return an empty string.
 - Extract the last two digits of the price to represent cents.
 - Extract the remaining digits to represent dollars.
 - Add commas to separate every three digits in the dollars part (e.g., 2000 becomes 2,000).
 - Concatenate the dollars, decimal points and cents.
 - It should also display an "Add to Cart" button.
- When the "Add to Cart" button is clicked (see Figure q2b):
 - The quantity displayed next to the shopping cart icon should increase accordingly. This can be done by calling a method that increments the value of the count data property by 1.
- When the added quantity reaches 10 (see Figure q2c):
 - The "Add to Cart" button should be disabled. This can be done by creating a computed property that returns true when the count value is greater than or equal to 10. Then, bind this computed property to the `disabled` attribute of the `<button>` element.

Instructions:

1. Navigate to the `q2` directory in your terminal.
2. Run the command `"pnpm install"` to install the required dependencies.
3. Run the command `"pnpm dev"` to run the app in your development server.
4. After installation, navigate to the `q2/src` directory.
5. Open the `App . vue` file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- The image file `cat.jpg` is provided in the `public/img` folder.
- All the necessary product data are provided in the Vue instance's data property.
- Explore the use of `.toLocaleString()`,
- The code to display the shopping cart icon has been provided.
- You may use Bootstrap for layout design and styling.

When the application first loads.

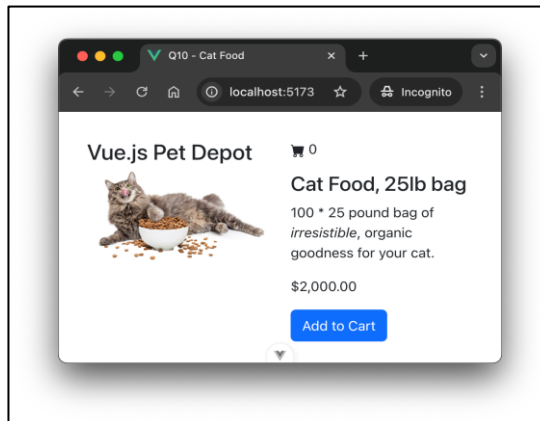


Figure q2a

When the “Add to Cart” button is clicked and the quantity increases.

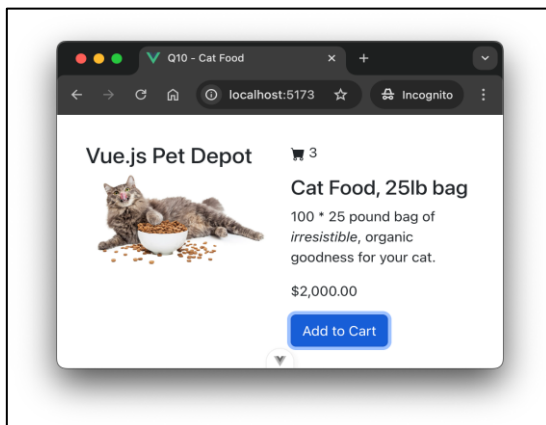


Figure q2b

When the quantity is 10, the “Add to Cart” button is disabled.

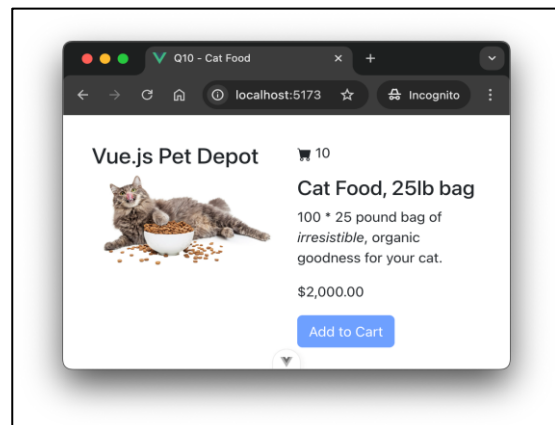


Figure q2c

Question 3 - Colourful words (*)

Background:

You are tasked with developing a colorful words web application. The application should transform the user's text input into a visually styled, colorful sentence (see Figures q3a and q3b).

Requirements:

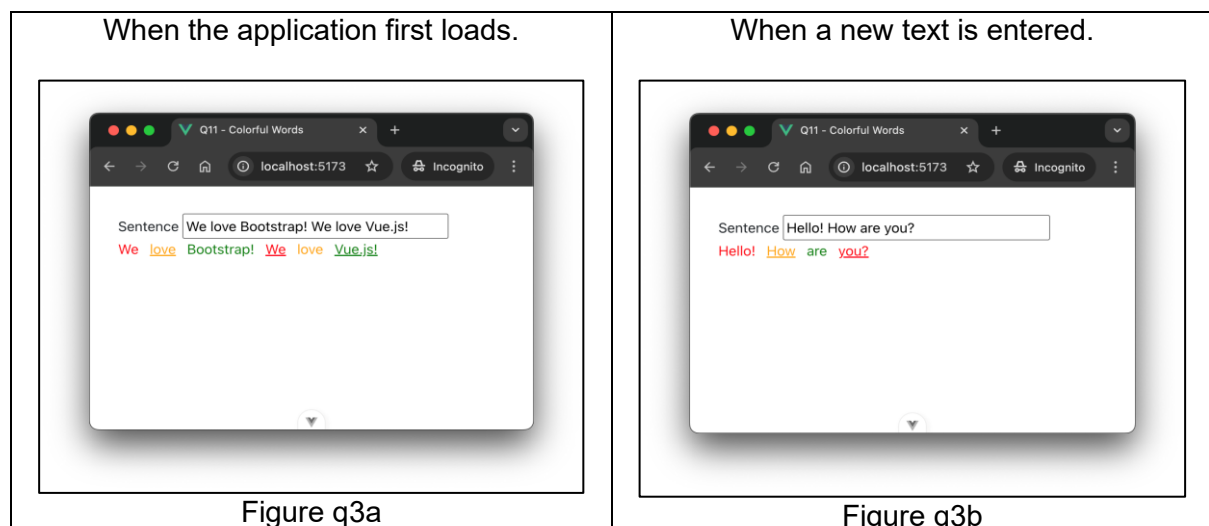
- When the application first loads (see Figure q3a):
 - It should display a text input field pre-loaded with the text from the sentence data property.
 - It should also display the content entered in the text input field on the second line with the following styles:
 - The 1st word is red, the 2nd is orange, and the 3rd is green.
 - Starting from the 4th word, the colors cycle in the same order: red, orange, green.
 - Words in even positions (2nd, 4th, 6th, etc.) are underlined.
 - Spaces between words should not be underlined.
- When new text is entered (see Figure q3b):
 - The displayed text should update automatically and apply the styles described above.

Instructions:

1. Navigate to the q3 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. After installation, navigate to the q3/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- You may assume that words are separated by a single space.



Question 4 - Flip coin (**)

Background:

You are tasked with developing a coin-flipping web application. The application should allow users to flip a customizable number of coins, and it should track and display the total number of flips made.

Requirements:

- When the application first loads (see Figure q4a):
 - It should display a text input field with a default value of 1, allowing the user to specify the number of coins to flip, along with a “Flip” button.
 - It should also display a single coin image (`tail.png`) and a text showing the total number of coin flips performed across all coins, accumulated over time. By default, this text should read “Flipped 0 times”.
- When “2” is entered into the text input field (see Figure q4b):
 - An additional coin image (`tail.png`) should be displayed, resulting in a total of two dynamically generated coin images. This can be done by detecting changes in the text input field using the `@change` event listener, which triggers a method to update the length of the `isHeadList` array. The number of `tail.png` coin images displayed will automatically adjust according to the updated length of `isHeadList`.
- When “2” remains in the text input field and the “Flip” button is clicked (see Figure q4c):
 - Each coin flips once, making the total number of flips 2. The text should update to “Flipped 2 times”. This can be done by calling a method that updates the flip count based on the number of flips performed.
- When “3” is entered into the text input field and the “Flip” button is clicked again (see Figure q4d):
 - Each coin flips once, adding 3 flips to the previous total of 2, for a cumulative total of 5 flips. The text should then read “Flipped 5 times”. This can be done by calling a method that updates the flip count based on the number of flips performed.

Instructions

1. Navigate to the `q4` directory in your terminal.
2. Run the command `pnpm install` to install the required dependencies.
3. Run the command `pnpm dev` to run the app in your development server.
4. After installation, navigate to the `q4/src` directory.
5. Open the `App.vue` file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- The coin image files `head.png` and `tail.png` are provided in the `public/img` folder.
- Explore the use of the created lifecycle hook and the `@change` event listener in Vue.

When the application first loads.

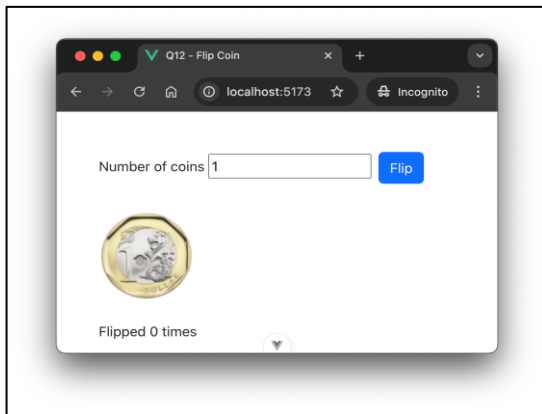


Figure q4a

When "2" is entered into the text input field.

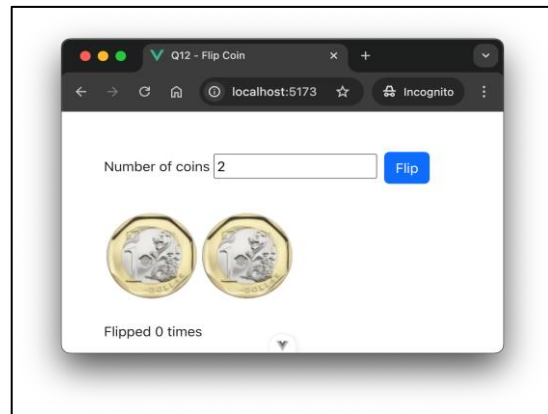


Figure q4b

When "2" remains in the text input field and the "Flip" button is clicked.

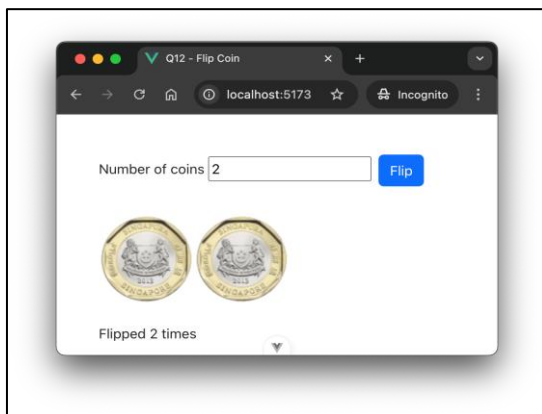


Figure q4c

When "3" is entered into the text input field and the "Flip" button is clicked again

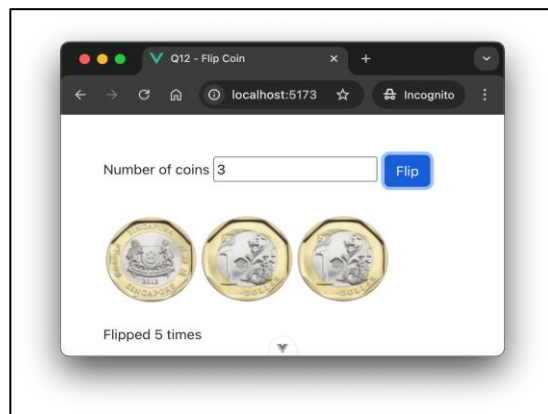


Figure q4d

Question 5 - Temperature (*)

Background:

You are tasked with developing a Singapore Weather Station web application. The application should retrieve "Air Temperature across Singapore" data from [data.gov.sg](https://data.gov.sg/dataset/realtime-weather-readings) (<https://data.gov.sg/dataset/realtime-weather-readings>), process it, and display the air temperature readings for various locations in Singapore.

Requirements:

- When the application first loads (see Figure q5a):
 - It should display the following:
 - The average temperature across all weather stations, rounded to two decimal places. This can be done by making an asynchronous Axios call to the link below and calculating the average from the returned temperature values.
 - <https://api-open.data.gov.sg/v2/real-time/api/air-temperature>
 - Timestamp of the temperature data, obtained from the API response.
 - A "Refresh" button that, when clicked, fetches the latest temperature data.
 - A table listing the temperatures recorded at all weather stations.

Instructions

1. Navigate to the q5 directory in your terminal.
2. Run the command "pnpm install" to install the required dependencies.
3. Run the command "pnpm dev" to run the app in your development server.
4. After installation, navigate to the q5/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.

When the application first loads.

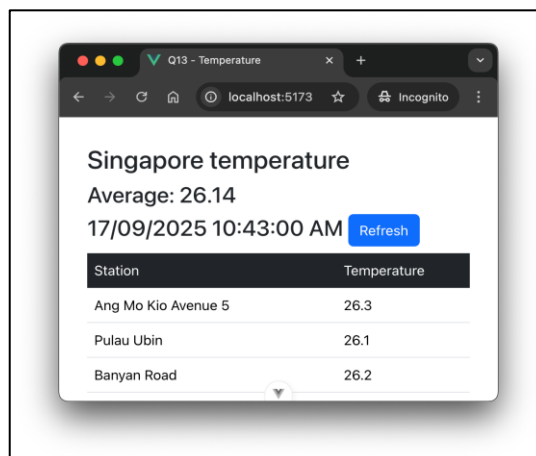


Figure q5a

Question 6 - Calculator (*/**/****)

Background:

You are tasked with developing a calculator web application. The application should perform basic arithmetic operations (+, −, ×, /) along with additional intuitive functions for user convenience.

Requirements:

Part A (*)

- When the application first loads (see Figure q6a):
 - It should display the following:
 - A display area for showing the mathematical expression and its result.
 - 18 buttons labelled: 0–9, +, −, ×, /, =, . (decimal point), AC (all clear), and BK (backspace).
 - The buttons should be arranged in rows of four, except for the AC and BK buttons, which may span a different number of columns as needed. This layout can be achieved by iterating over the buttons array from the component's data property using the v-for directive.

Part B (**)

Implement basic addition and subtraction functionality as follows:

- When any button (except "=", "AC", and "BK") is clicked:
 - The corresponding value is appended to the expression displayed; in the order the buttons are clicked (see Figure q6b).
 - If the first button clicked is an operator (+, −, ×, /), automatically insert 0 as the first operand (see Figure q6c).
- When the "=" button is clicked:
 - Evaluate the expression and display the result if it is valid (see Figure q6d).
 - If the expression is invalid, display "ERR" (see Figure q6e).
- When the "AC" button is clicked:
 - Clear both the expression and result displays.

Part C (**)

- After a result is displayed whether it is a valid result or "ERR":
 - Clicking any non-operator button starts a new expression, replacing the previous one.
 - Clicking an operator button starts a new expression using the displayed result as the first operand, followed by the operator.

Part D (**)

- When the "BK" button is clicked:
 - If a result is currently displayed, clicking "BK" will reset the calculator by clearing both the expression and result displays.
 - If no result is displayed, clicking "BK" removes the last character from the current expression.

Part E (*)**

- Implement multiplication and division functionality (see Figures q6f).
- Support mixed operations within a single expression using +, −, ×, and /.
 - Ensure proper operator precedence, where multiplication (×) and division (/) take precedence over addition (+) and subtraction (−) (see Figures q6g).

Instructions

1. Navigate to the q6 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. After installation, navigate to the q6/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.

When the application first loads.

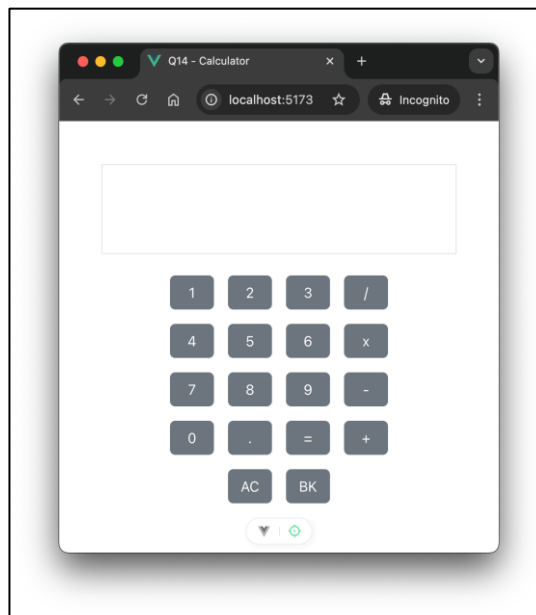


Figure q6a

When the first button clicked is a number.

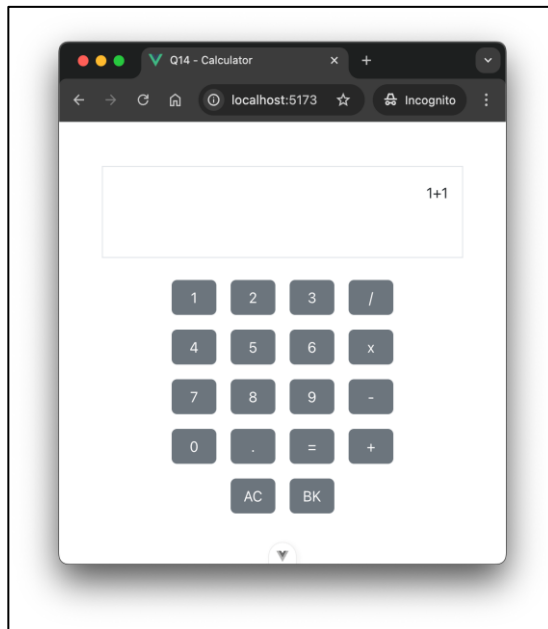


Figure q6b

When the first button clicked is an operator.

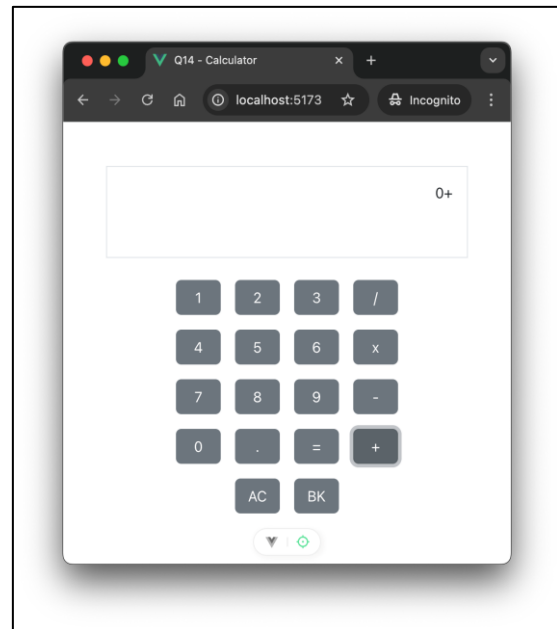


Figure q6c

When a valid expression is entered.

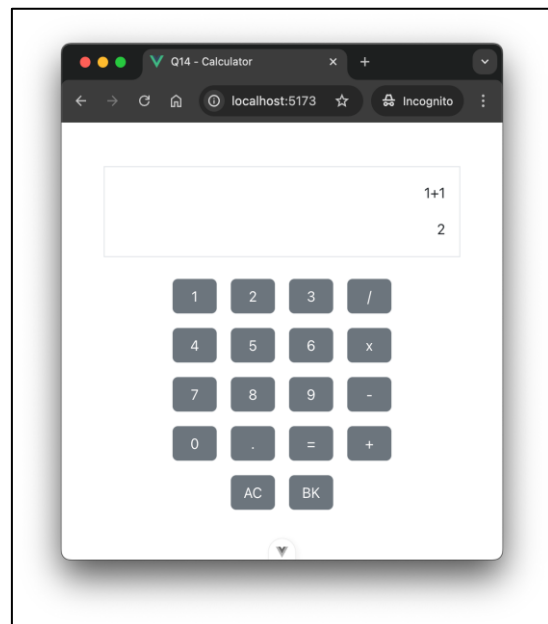


Figure q6d

When an invalid expression is entered.

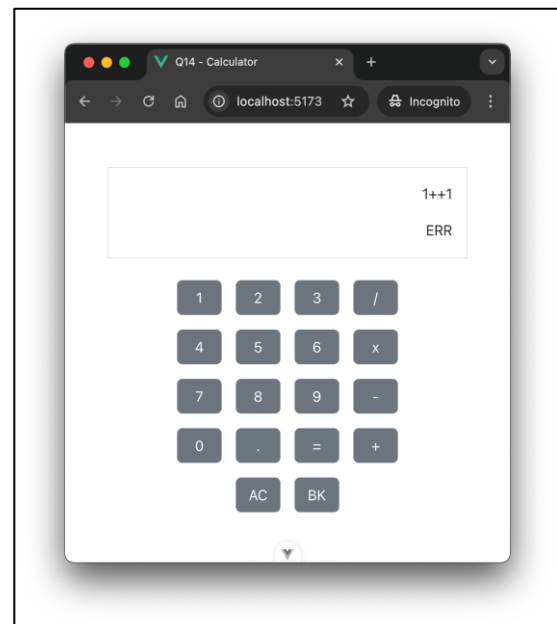


Figure q6e

When a valid expression with a multiplication operator is entered.

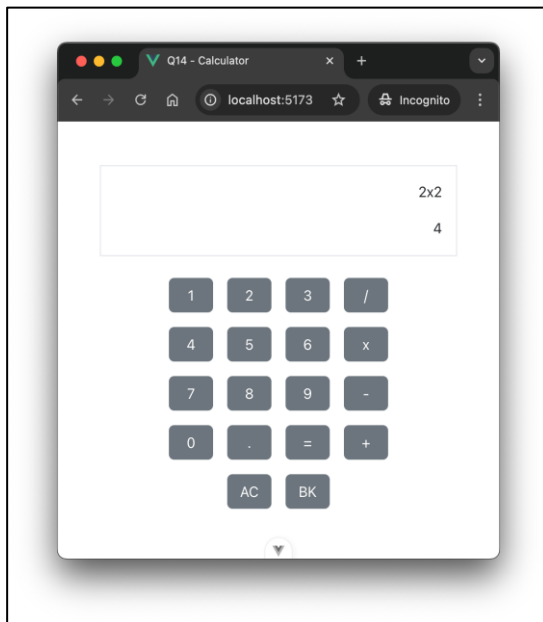


Figure q6f

When mixed operations within a single expression are entered.

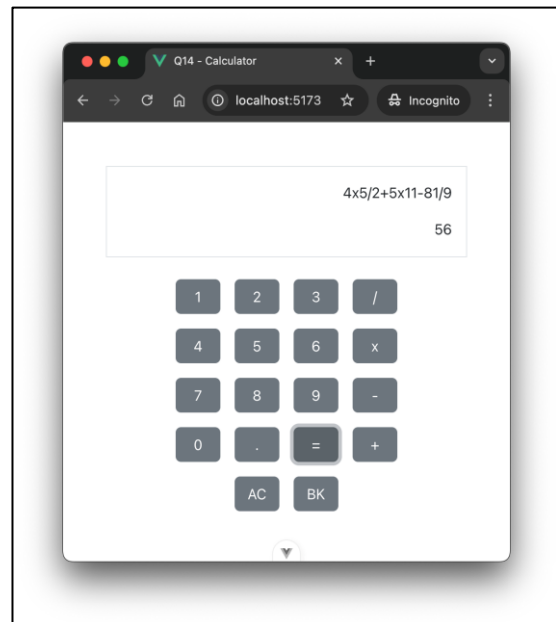


Figure q6g

Question 7 - Monster Slayer 2 (***)

Background:

You are tasked with developing a Monster Slayer 2 game. The core gameplay is similar to the version demonstrated in the lecture slides, but with modified requirements for Player and Monster Strength, Attack, Special Attack, and Heal mechanics.

Requirements:

- When the application first loads (see Figure q7a):
 - It should display the following:
 - Images of the player (player.png) and the monster (monster.png).
 - Two green health bars representing the health of both the player and the monster. Each should display 100% by default and update to reflect the starting strength selected by the user.
 - Two health selection drop-down menus, allowing the user to choose the starting strength for both the player and the monster either Full Strength (100) or Half Strength (50).
 - A “START NEW GAME” button to begin the game.
 - A game battle log section that records the progress and actions throughout the game.
- When the “START NEW GAME” button is clicked (see Figure q7b):
 - It should display the following:
 - Four buttons labelled “ATTACK”, “SPECIAL ATTACK”, “HEAL”, and “GIVE UP”. These buttons can be generated dynamically by iterating over the buttonDetails array from the component’s data property using the v-for directive.
 - The same game battle log section created earlier that records the progress and actions throughout the game.
- When the “ATTACK” button is clicked:
 - The player attacks the monster, dealing a random integer amount of damage up to a maximum of 30. Afterward, the monster attacks the player with damage up to a maximum of 30.
 - The green health bars for both the player and the monster update accordingly, and both attack events (including damage dealt) are recorded in the battle log (see Figure q7c).
 - If the monster’s health drops to 60 or below, there is a 50% chance it will heal itself by a random integer amount instead of attacking (see Figure q7d).
- When the “SPECIAL ATTACK” button is clicked:
 - The player attacks the monster, dealing a random integer amount of double damage.
 - After using a special attack, a 2-round cooldown is applied, during which the special attack button is hidden, and the player can only choose Attack or Heal (see Figure q7e).

- When the “HEAL” button is clicked:
 - The player restores life by a random integer amount up to a maximum of 30. Afterward, the monster takes its turn and performs its move accordingly (see Figure q7f).
- When the “GIVE UP” button is clicked:
 - The game ends (see Figure q7g).

Instructions

1. Navigate to the q7 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. After installation, navigate to the q7/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 800px.
- The image files player.png and monster.png are provided in the public/img folder.

When the application first loads.

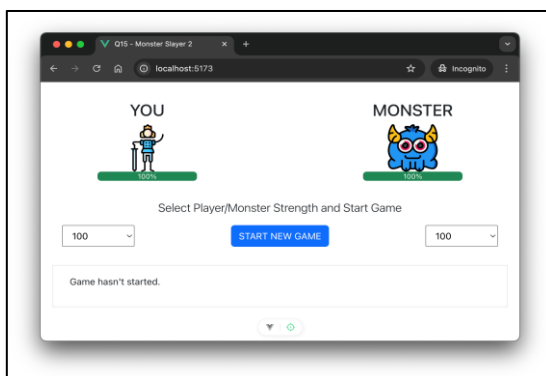


Figure q7a

When the “START NEW GAME” button is clicked.

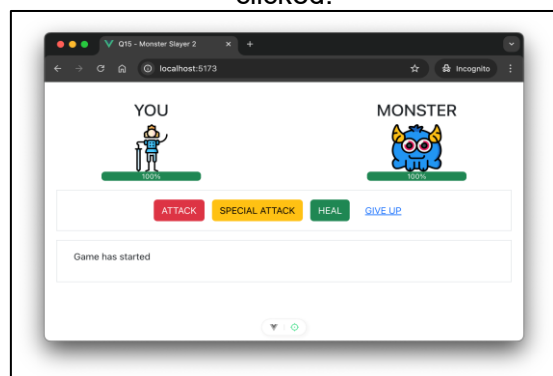


Figure q7b

When the “ATTACK” button is clicked, the player attacks first, followed by an immediate counterattack from the monster.

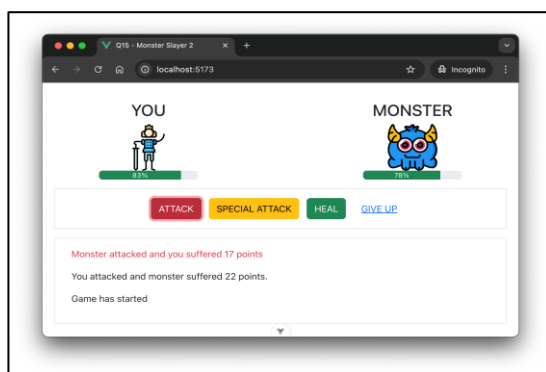


Figure q7c

When the “ATTACK” button is clicked, the player attacks first, followed by the monster immediately healing itself.

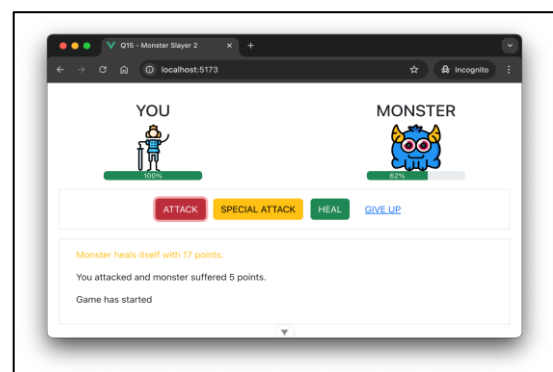


Figure q7d

When the “SPECIAL ATTACK” button is clicked, the player attacks with double damage and enters a 2-round cooldown period during which the special attack option is unavailable.

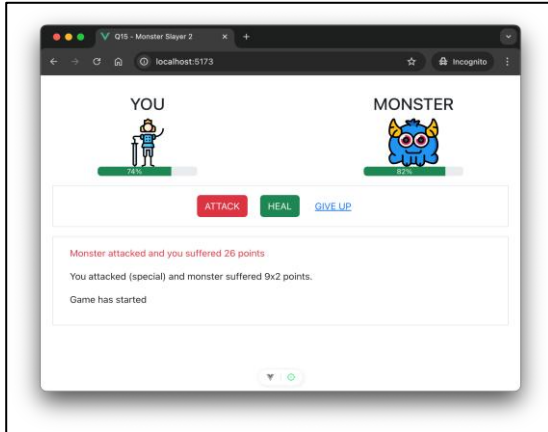


Figure q7e

When the “HEAL” button is clicked, the player heals himself, followed by the monster taking its turn to perform its move accordingly.

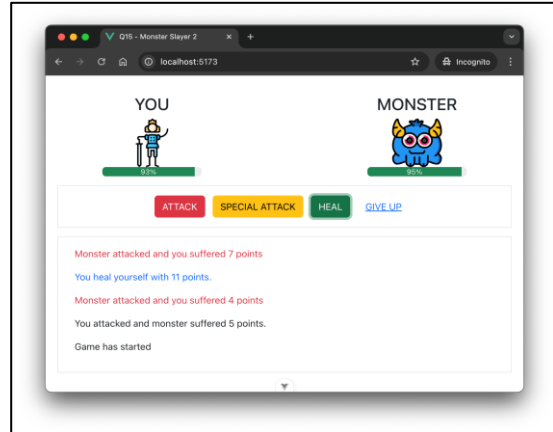


Figure q7f

When the “GIVE UP” button is clicked, the game ends.

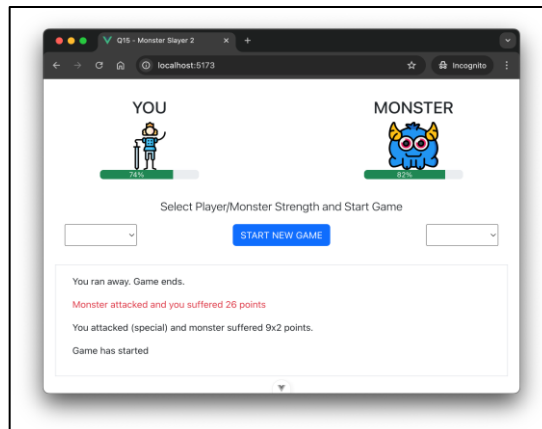


Figure q7g